

PES UNIVERSITY

100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering

(AI & ML)

B. Tech. CSE - 5th Semester

Aug - Dec 2026

UE24CS351A

DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

SkillSync: Smart Career & Skill Recommendation Portal

Submitted by:

SRN	Name	Section
PES1UG24AM194	Pradyumn Bidari	5D
PES1UG24AM201	Prathyush Gowda	5D

Abstract

SkillSync is an academic, database-driven web application that evaluates the suitability of student skill profiles for different career paths. Many students know their current technical skills but lack a structured way to compare those proficiencies with career-specific requirements. The application links students, skills, careers and required proficiency levels to calculate an explainable career-readiness percentage, list missing skills, prioritize those gaps and estimate learning hours.

The implementation uses Next.js, React and TypeScript for the interface and server-side routes, MySQL for structured relational data, and MongoDB Atlas for flexible mentor feedback. The MySQL schema contains twelve base tables, including associative relations for student skills, career skills and mentor assignments. Three SQL views summarize career readiness, skill demand and learning activity. The application additionally provides authenticated student, mentor and administrator roles, database reports, role-based authorization, parameterized queries, and transactional skill updates.

A verified SQL demonstration temporarily lowered Python proficiency and changed ML Engineer readiness from 73.1% to 67.4%, while estimated learning effort increased from 96 to 108 hours. The original results returned after ROLLBACK. This supports the conclusion that recommendations are calculated from database values rather than fixed display text. Career weights and learning-hour assumptions remain illustrative and do not constitute a trained machine-learning model. The independent, non-LLM originality requirement requires separate student-authored design and evidence.

Keywords

DBMS; MySQL; MongoDB Atlas; Next.js; Relational Schema; Primary Key; Foreign Key; SQL JOIN; Stored View; Trigger; Transaction; Rule-Based Career Recommendation; Skill Gap Analysis.

Table of Contents

Sl. No.	Topic	Page No.
1	Introduction - Problem Statement	4
2	User Requirements Specification - Tech Stack	5
3	E-R Model	6
4	Relational Model	8
5	SQL DDL Statements	10
6	SQL DML Statements	12
7	NoSQL / Vector Database Commands	13
8	Results - Resulting Tables' Screenshots	14
9	Next.js Project Tree Structure	16
10	Sample Screenshots of Application Code and Demo	17
11	Evaluation Rubrics - Completion Status	19

1. Introduction

1.1 Problem Statement

Students face difficulty choosing realistic career directions because skill assessments, career requirements, mentor advice and learning records often remain scattered across different tools. A conventional list of recommended careers does not explain which skills are missing, how important those skills are or how much learning may be needed. SkillSync addresses this by maintaining structured student and career data and producing transparent, dynamically calculated recommendations.

1.2 Objectives

- Create a normalized relational data model with students, careers, skills, and authenticated user records.
- Compute weighted career readiness from existing and required skill proficiency levels.
- Identify missing proficiency levels and rank skill gaps by requirement importance.
- Store projects, certifications, learning hours and recommendation-history snapshots.
- Record mentor feedback in a NoSQL collection and retrieve it through authorized APIs.
- Demonstrate core DBMS concepts, reporting, views, triggers, stored routines and transaction control.

1.3 Scope of the Prototype

The system runs as a local academic web application connected to a MySQL database and a cloud-hosted MongoDB Atlas collection. The sample students, career requirements, skill importance values and hours-per-level estimates support functional demonstration rather than validated occupational prediction. It does not use trained ML models, pgvector or semantic embedding retrieval.

1.4 Basic Workflow

Student signs in -> enters proficiency levels -> server retrieves relational data -> SQL view calculates career readiness -> student sees ranked careers and missing skills -> mentor reviews assigned student and stores guidance -> administrator views reports and manages assignments.

2. User Requirements Specification

2.1 Functional Requirements

Actor	Requirement
Student	Sign in; edit profile and skill levels; view readiness, skill gaps, projects, certificates, progress and mentor feedback.
Mentor	Access assigned students and write/update feedback with prioritized recommended skills.
Admin	View aggregated SQL reports, manage mentor assignments, inspect accounts, update career-skill requirements.
System	Validate request payloads, enforce server-side role permissions, persist history and audit relevant changes.

2.2 Non-Functional Requirements

Integrity through PK, FK, UNIQUE and CHECK constraints; maintainability through modular Next.js routes and reusable database clients; security via hashed credentials, authenticated role checks and server-side environment variables; consistent updates through MySQL transactions; and responsive tabular reports. Production deployment, penetration testing and scalability benchmarks are outside the evaluated scope.

2.3 Technology Stack and Justification

Layer	Technology	Why selected
Frontend	Next.js 16 / React 19 / TypeScript	Reusable pages and interactive student/mentor/admin interfaces.
Backend	Next.js server routes / Node.js	API endpoints execute SQL or MongoDB operations on server side.
Relational DB	MySQL 8 + mysql2/promise	Joins and foreign keys support strongly related student/skill/career data.
NoSQL DB	MongoDB Atlas + mongodb driver	Flexible nested mentor feedback and recommended skill arrays.
Authentication	Auth.js credentials/JWT + bcryptjs	Distinct student, mentor and admin roles; hashed passwords.
Validation	Zod	Checks request shape, IDs and proficiency ranges.

2.4 Why MongoDB Instead of a Vector Database?

Mentor feedback is semi-structured: a comment can accompany a list of skills with different priorities and additional progress metadata. MongoDB documents naturally represent this variable shape, whereas a vector store is unnecessary for a rule-based, SQL-weighted recommender. SkillSync uses NoSQL document retrieval/update, not vector similarity search.

3. E-R Model

3.1 List of Entities (Weak Ones Identified)

Strong entity types: STUDENT, SKILL, CAREER, PROJECT, CERTIFICATION, APP_USER, LEARNING_PROGRESS, RECOMMENDATION_HISTORY and AUDIT_LOG. The remaining dependent associative entities are STUDENT_SKILLS, CAREER_SKILLS and MENTOR_ASSIGNMENT. The latter use composite keys made from parent identifiers and are drawn as dependent/associative entities in the conceptual illustration.

Formal Chen-notation note: a weak entity is identified through an owner relationship and a partial key. In the physical schema, LEARNING_PROGRESS and RECOMMENDATION_HISTORY have their own surrogate primary keys; they are not strictly weak entities. Orange double-bordered shapes in the presentation illustration indicate dependency only, and the relational schema on the following pages is authoritative.

3.2 Non-Simple, Multivalued and Derived Attributes

Attribute category	Actual SkillSync example
Composite / nested	MongoDB mentor feedback: recommended_skills[] contains skill and priority pairs.
Multivalued	A mentor document may hold multiple recommended skill objects in one array.
Derived	readiness_percent, missing_levels, priority_score and estimated_learning_hours are computed from SQL rules; history stores snapshots.
JSON	audit_logs.details stores flexible structured audit details in MySQL JSON type.

3.3 Key Relationships

STUDENT M:N SKILL through STUDENT_SKILLS (proficiency); CAREER M:N SKILL through CAREER_SKILLS (required_level, importance); STUDENT 1:N PROJECT/CERTIFICATION/LEARNING_PROGRESS/RECOMMENDATION_HISTORY; APP_USER M:N STUDENT through MENTOR_ASSIGNMENTS; optional STUDENT 0..1 : 0..1 APP_USER login link; APP_USER 1:N AUDIT_LOGS when actor is present.

3.4 ER Diagram Snapshot (Chen Style)

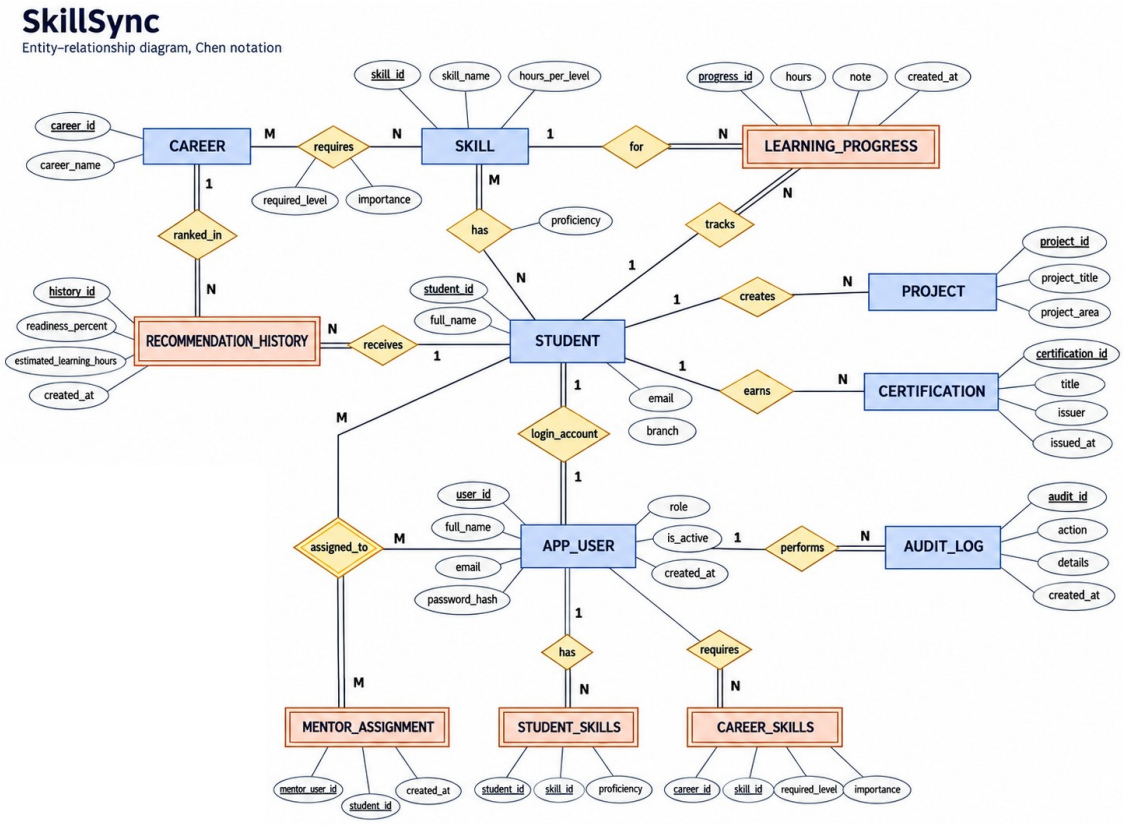


Figure 1. SkillSync Chen-style conceptual ER illustration. Dependency styling is schematic; physical SQL keys are documented in Section 4.

In this conceptual drawing, rectangles indicate entities, diamonds indicate relationships and ellipses indicate attributes. Double outlines are used to discuss dependency; the corresponding relational keys and optionality must be checked against the physical schema diagrams in Section 4.

4. Relational Model

4.1 List of Relations

Relation	Key attributes and selected fields
students	student_id (PK); full_name; email (UQ); branch
skills	skill_id (PK); skill_name (UQ); hours_per_level
careers	career_id (PK); career_name (UQ)
student_skills*	student_id + skill_id (composite PK, FK); proficiency
career_skills*	career_id + skill_id (composite PK, FK); required_level; importance
projects	project_id (PK); student_id (FK); project_title; project_area
app_users	user_id (PK); email (UQ); role; student_id (FK, nullable)
mentor_assignments*	mentor_user_id + student_id (composite PK, FK); created_at
certifications	certification_id (PK); student_id (FK); title; issuer; issued_at
learning_progress	progress_id (PK); student_id (FK); skill_id (FK); hours; note
recommendation_history	history_id (PK); student_id (FK); career_id (FK); readiness_percent
audit_logs	audit_id (PK); actor_user_id (FK, nullable); action; details (JSON)

* Relations created when mapping conceptual many-to-many relationships or mentor assignment into SQL tables. They are junction/associative relations rather than independent core entity types in the conceptual model.

4.2 Schema Diagram - Core Tables

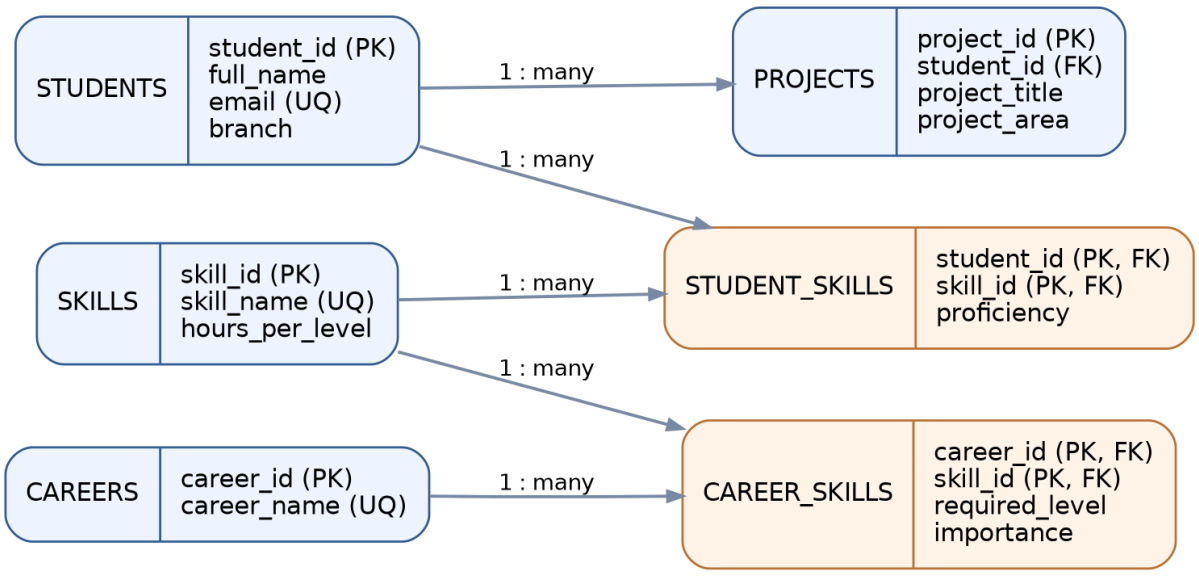


Figure 2. Light-background relational schema for the six core tables; orange boxes are junction relations.

4. Relational Model (continued)

4.3 Schema Diagram - Extended Tables

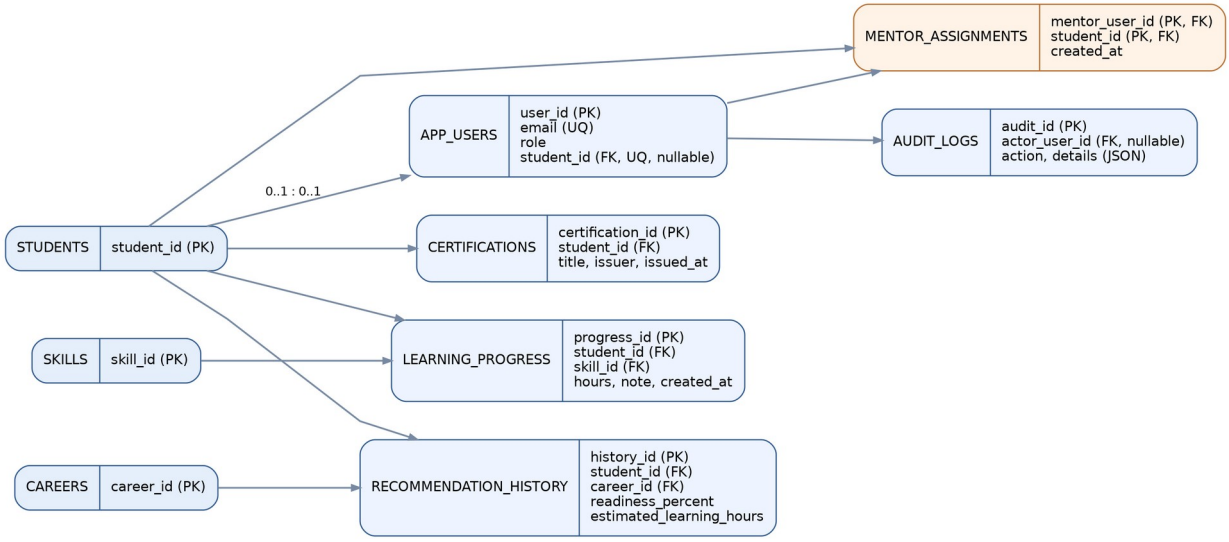


Figure 3. Light-background relational schema for accounts, assignments, learning, certifications, history and audit.

4.4 Key Constraints and Integrity

The table app_users has a unique email and an optional unique student_id foreign key. mentor_assignments uses a composite primary key across its two foreign keys. learning_progress requires a positive hours value. recommendation_history references both students and careers. Most dependent student records use ON DELETE CASCADE; app_users.student_id and audit_logs.actor_user_id use ON DELETE SET NULL. The server uses SQL parameter placeholders for values.

4.5 Normalization

Student details, skill definitions and career names are stored once in their respective relations, avoiding name repetition in each proficiency or requirement row. Many-to-many links are represented through junction relations. This is consistent with common normalization goals; a formal proof of each relation's highest normal form is outside this report.

5. SQL DDL Statements

DDL statements define relational objects and constraints. The source file scripts/setup-db.mjs checks for the six existing core tables and creates six additional tables without deleting previous data. The SQL below is a representative extract of the project implementation.

5.1 Table Creation and Constraints

```
CREATE TABLE IF NOT EXISTS mentor_assignments (
  mentor_user_id INT NOT NULL,
  student_id INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (mentor_user_id, student_id),
  FOREIGN KEY (mentor_user_id) REFERENCES app_users(user_id)
    ON DELETE CASCADE,
  FOREIGN KEY (student_id) REFERENCES students(student_id)
    ON DELETE CASCADE
) ENGINE=InnoDB;


CREATE TABLE IF NOT EXISTS learning_progress (
  progress_id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT NOT NULL, skill_id INT NOT NULL,
  hours DECIMAL(8,2) NOT NULL,
  note VARCHAR(300) NOT NULL DEFAULT '',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_progress_hours CHECK (hours > 0),
  FOREIGN KEY (student_id) REFERENCES students(student_id)
    ON DELETE CASCADE,
  FOREIGN KEY (skill_id) REFERENCES skills(skill_id)
    ON DELETE CASCADE
) ENGINE=InnoDB;
```

5.2 View Definition (Career Readiness)

```
CREATE VIEW v_career_readiness AS
SELECT s.student_id, s.full_name, c.career_id, c.career_name,
 ROUND(100.0 * SUM(cs.importance *
   LEAST(COALESCE(ss.proficiency,0)/cs.required_level,1.0)) /
   NULLIF(SUM(cs.importance),0),1) AS readiness_percent,
 SUM(GREATEST(cs.required_level-COALESCE(ss.proficiency,0),0)
    * sk.hours_per_level) AS estimated_learning_hours
FROM students s CROSS JOIN careers c
JOIN career_skills cs ON cs.career_id=c.career_id
JOIN skills sk ON sk.skill_id=cs.skill_id
LEFT JOIN student_skills ss ON ss.student_id=s.student_id
 AND ss.skill_id=cs.skill_id
GROUP BY s.student_id,s.full_name,c.career_id,c.career_name;
```

5. SQL DDL Statements (continued)

5.3 Indexes, Trigger and Stored Routine

```
CREATE INDEX idx_history_student_created
 ON recommendation_history(student_id,created_at);
CREATE INDEX idx_progress_student_created
 ON learning_progress(student_id,created_at);


CREATE TRIGGER trg_skillsync_skill_update
AFTER UPDATE ON student_skills FOR EACH ROW
INSERT INTO audit_logs(action,details)
VALUES('SKILL_UPDATED', JSON_OBJECT(
 'student_id',NEW.student_id, 'skill_id',NEW.skill_id,
 'old_level',OLD.proficiency,'new_level',NEW.proficiency));


CREATE PROCEDURE sp_student_career_report(IN p_student_id INT)
SELECT career_name,readiness_percent,estimated_learning_hours
FROM v_career_readiness
WHERE student_id=p_student_id ORDER BY readiness_percent DESC;


CREATE FUNCTION fn_missing_levels(
 current_level INT, required_level INT)
RETURNS INT DETERMINISTIC NO SQL
RETURN GREATEST(required_level-current_level,0);
```

The setup also defines v_skill_demand and v_learning_hours, INSERT/DELETE audit triggers and an audit JSON field. Optional creation of triggers or routines may be skipped by the setup if the current MySQL account lacks privileges; use SHOW TRIGGERS and SHOW PROCEDURE/FUNCTION STATUS to verify actual installation.

5.4 Inspection Commands

```
USE skillsync_review;
SHOW FULL TABLES;
SHOW CREATE TABLE student_skills;
SHOW INDEX FROM recommendation_history;
SHOW TRIGGERS LIKE 'student_skills';
SHOW PROCEDURE STATUS WHERE Db='skillsync_review';
```

6. SQL DML Statements

6.1 INSERT / UPDATE / DELETE in the Student Skills API

When a student saves proficiency, the backend validates an integer from 0 to 5, starts a transaction and either deletes a zero-level record or performs a parameterized INSERT ... ON DUPLICATE KEY UPDATE. It then saves recommendation snapshots and commits, or rolls back on error.

```
START TRANSACTION;
INSERT INTO student_skills(student_id,skill_id,proficiency)
VALUES (?,?,?)
ON DUPLICATE KEY UPDATE proficiency=VALUES(proficiency);
INSERT INTO recommendation_history
 (student_id,career_id,readiness_percent,estimated_learning_hours)
SELECT student_id,career_id,readiness_percent,
 estimated_learning_hours FROM v_career_readiness
WHERE student_id=?;
COMMIT;
-- Level zero path in the API:
DELETE FROM student_skills WHERE student_id=? AND skill_id=?;
```

6.2 JOIN and Grouped Reports

```
SELECT s.full_name,sk.skill_name,ss.proficiency
FROM student_skills ss
JOIN students s ON s.student_id=ss.student_id
JOIN skills sk ON sk.skill_id=ss.skill_id LIMIT 10;

SELECT s.branch,COUNT(p.project_id) AS project_count
FROM students s LEFT JOIN projects p
 ON p.student_id=s.student_id GROUP BY s.branch;
```

6.3 Nested Query (Demonstration SQL)

```
SELECT full_name,career_name,readiness_percent
FROM v_career_readiness
WHERE readiness_percent > (
 SELECT AVG(readiness_percent) FROM v_career_readiness
) LIMIT 10;
```

Nested and CTE queries are SQL demonstration examples, not claims of advanced queries executed by every website page. The project source directly confirms JOINs and aggregation in its reports.

6.4 Common Table Expression (Optional Rubric Demonstration)

```
WITH career_avg AS (
 SELECT career_name,AVG(readiness_percent) AS avg_score
 FROM v_career_readiness GROUP BY career_name
)
SELECT career_name,ROUND(avg_score,1) AS average_readiness
FROM career_avg ORDER BY avg_score DESC;
```

7. NoSQL / Vector Database Commands

7.1 MongoDB Database and Collection

Database: skillsync_nosql. Collection: mentor_feedback. MongoDB Atlas stores flexible feedback, a nested recommended_skills array and progress metadata. The implementation uses the MongoDB Node.js driver and document operations; it does not implement vector similarity or a vector index.

7.2 Document Structure (Illustrative)

```
{
  "student_id": 1,
  "student_name": "Example Student",
  "mentor_name": "Example Mentor",
  "feedback": "Improve Statistics and complete an ML project",
  "recommended_skills": [
    {"skill": "Statistics", "priority": "High"},
    {"skill": "TensorFlow", "priority": "Medium"}
  ],
  "updated_at": ISODate("2026-11-01T00:00:00Z")
}
```

7.3 Commands Used by the Application

```
// Read feedback by student id (or legacy student name)
db.mentor_feedback.findOne({student_id: 1});

// Authorized mentor/admin updates feedback, with upsert
db.mentor_feedback.updateOne(
 {student_id: 1},
 {$set: {feedback: "Revise Statistics",
        mentor_name: "Example Mentor",
        recommended_skills: [
         {skill: "Statistics", priority: "High"}]}},
 {upsert: true}
);
```

The API actually calls collection.findOne() and collection.updateOne(..., {upsert: true}), and verifies permissions through MySQL mentor assignments before allowing access. A MongoDB delete action is not evidenced in the present application. Cross-database foreign-key enforcement is not available.

8. Results

8.1 Recorded SQL Before/After Verification

A live SQL transaction temporarily lowered the Python proficiency for one student and queried the ML Engineer readiness view. The following outputs were reported during the project demonstration:

Stage	ML Engineer readiness	Estimated learning hours
BEFORE	73.1%	96 hours
AFTER	67.4%	108 hours
RESTORED after ROLLBACK	73.1%	96 hours

Readiness decreased by 5.7 percentage points and the estimated learning time increased by 12 hours. ROLLBACK restored the original values. This verifies dynamic SQL-derived calculations and transaction restoration for this test.

```
    -> WHERE student_id = @sid AND career_id = @cid;
+--------+-------------+-------------------+-------------------------+
| stage | career_name | readiness_percent | estimated_learning_hours |
+--------+-------------+-------------------+-------------------------+
| AFTER | ML Engineer |              67.4 |                     108 |
+--------+-------------+-------------------+-------------------------+
1 row in set (0.00 sec)

mysql>
mysql> -- RESTORE ORIGINAL DATA
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT 'RESTORED' AS stage,
    ->        career_name,
    ->        readiness_percent,
    ->        estimated_learning_hours
    -> FROM v_career_readiness
    -> WHERE student_id = @sid AND career_id = @cid;
+----------+-------------+-------------------+-------------------------+
| stage    | career_name | readiness_percent | estimated_learning_hours |
+----------+-------------+-------------------+-------------------------+
| RESTORED | ML Engineer |              73.1 |                      96 |
+----------+-------------+-------------------+-------------------------+
1 row in set (0.00 sec)

mysql>
mysql>
```

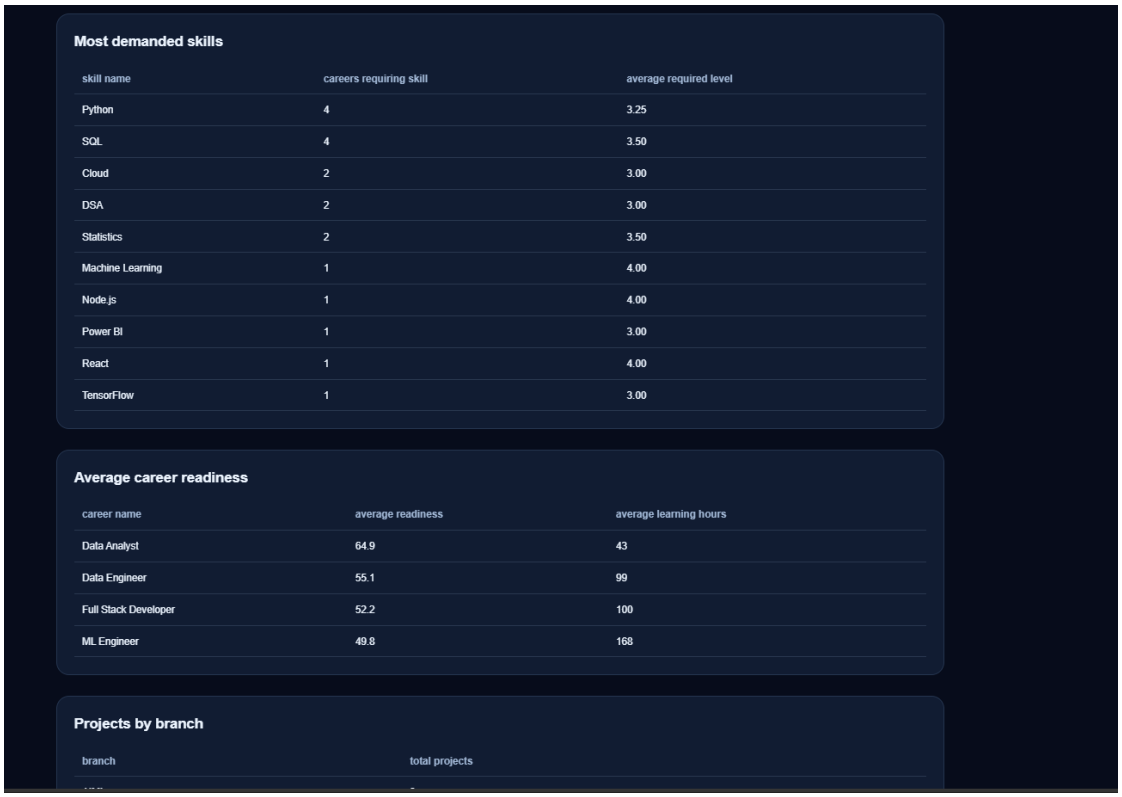
Figure 4. Actual MySQL terminal output captured during the SQL AFTER and RESTORED test.

8.2 Resulting Tables and Database Connectivity

At the recorded verification point, the script showed 121 career score rows, account roles (1 student, 5 mentors, 1 admin), and one mentor-feedback document. These are a point-in-time test output and not a current live count.

8. Results (continued)

8.3 Application SQL Report Tables



Most demanded skills

skill name	careers requiring skill	average required level
Python	4	3.25
SQL	4	3.50
Cloud	2	3.00
DSA	2	3.00
Statistics	2	3.50
Machine Learning	1	4.00
Node.js	1	4.00
Power BI	1	3.00
React	1	4.00
TensorFlow	1	3.00

Average career readiness

career name	average readiness	average learning hours
Data Analyst	64.9	43
Data Engineer	55.1	99
Full Stack Developer	52.2	100
ML Engineer	49.8	168

Projects by branch

branch	total projects

Figure 5. Admin report table results: demanded skills, average career readiness and projects by branch.

The screenshot is an application-generated report table rather than a direct MySQL SHOW TABLES screenshot. A direct SQL client table inventory and a few SELECT outputs should be captured before final mentor submission if required strictly by the rubric.

8.4 Calculation Formulas

```
Readiness (%) =
 100 * SUM[importance * MIN(current_level/required_level,1)]
     / SUM[importance]
Missing levels = MAX(required_level - current_level, 0)
Priority score = importance * missing_levels
Learning hours = SUM[missing levels * hours_per_level]
```

The coefficients are stored in career_skills and skills. They allow the system to explain why a career is ranked higher or which skills need improvement. These assumptions are illustrative rather than evidence of predictive accuracy.

9. Next.js Project Tree Structure

The project source follows the Next.js App Router layout. The outline below omits generated .next and node_modules directories.

```
skillsync-complete/
|-- app/
|   |-- api/
|   |   |-- auth/[...nextauth]/route.ts
|   |   |-- readiness/route.ts
|   |   |-- gaps/route.ts
|   |   |-- skills/route.ts
|   |   |-- mentor-feedback/route.ts
|   |   |-- projects/route.ts
|   |   |-- certifications/route.ts
|   |   |-- progress/route.ts
|   |   |-- history/route.ts
|   |   |-- profile/route.ts
|   |   |-- careers/route.ts
|   |-- dashboard/  skill-gaps/  skills/  profile/
|   |-- projects/  certifications/  progress/
|   |-- history/  mentor/  mentor-feedback/  admin/
|   `-- login/
|-- components/
|   |-- student-dashboard.tsx
|   |-- gap-explorer.tsx
|   |-- mentor-workspace.tsx
|   `-- admin-workspace.tsx
|-- lib/
|   |-- db.ts            # mysql2 connection pool
|   |-- mongodb.ts       # MongoDB Atlas client
|   `-- permissions.ts   # server-side authorization
|-- scripts/
|   |-- setup-db.mjs
|   |-- verify.mjs
|   |-- create-user.mjs
|   `-- assign-mentor.mjs
|-- sql/advanced_demos.sql
|-- docs/ER_Extended_Mermaid.md
|-- auth.ts
|-- .env.example
`-- package.json
```

Primary execution command: npm run dev. Database-check command: npm run verify. Sensitive credentials belong in a local .env.local file and must never be published in the report or version-control repository.

10. Few Sample Screenshots of Application Code and Demo

10.1 Skill Gap Analyzer - Student View

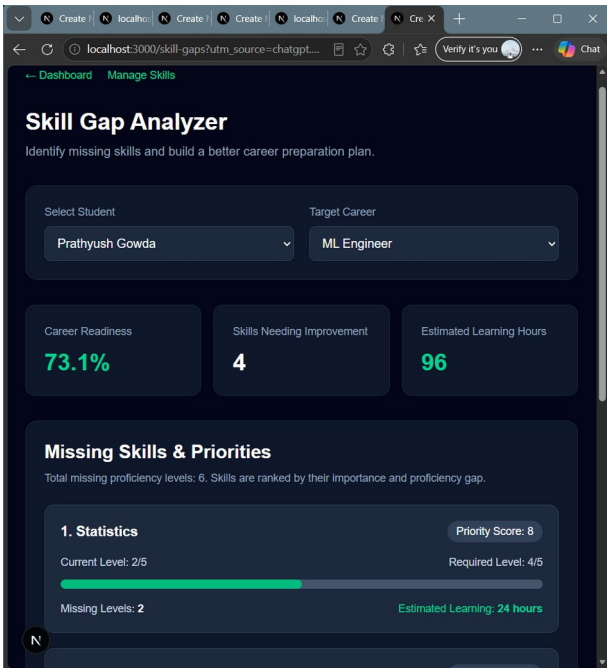


Figure 6. Skill Gap Analyzer shows ML Engineer readiness, number of gaps and estimated learning effort.

10.2 Mentor Feedback - Student View

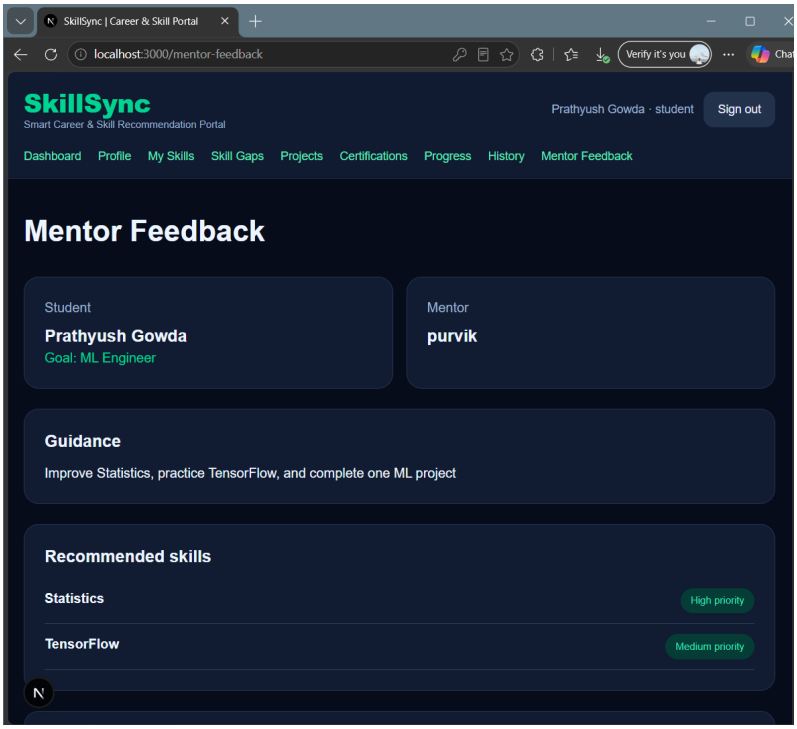


Figure 7. Mentor guidance and skill recommendations retrieved from the MongoDB collection.

10. Application Code and Demo (continued)

10.3 Backend Skills API - Transactional Update Excerpt

```
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();
  await conn.execute(
   "INSERT INTO student_skills (student_id,skill_id,proficiency)"
   + " VALUES (?,?,?) ON DUPLICATE KEY UPDATE"
   + " proficiency=VALUES(proficiency)",
   [u.studentId, skillId, proficiency]
  );
  await conn.execute(
   "INSERT INTO recommendation_history"
   + " (student_id,career_id,readiness_percent,"
   + " estimated_learning_hours)"
   + " SELECT student_id,career_id,readiness_percent,"
   + " estimated_learning_hours FROM v_career_readiness"
   + " WHERE student_id=?", [u.studentId]
  );
  await conn.commit();
} catch(e) { await conn.rollback(); throw e; }
finally { conn.release(); }
```

The excerpt illustrates the nonzero proficiency update path from app/api/skills/route.ts. The production code separately handles proficiency 0 using DELETE; both branches are within a transaction.

10.4 Authentication and Backend Data Flow

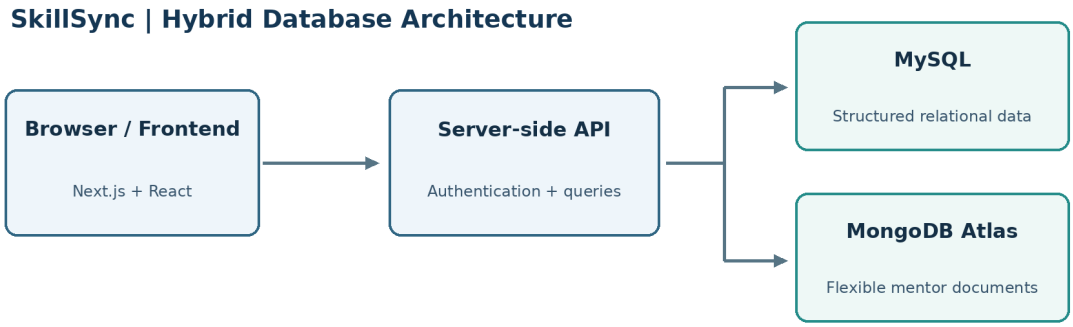


Figure 8. Backend request flow: browser -> server-side Next.js API -> MySQL and MongoDB Atlas.

Credential login uses Auth.js, bcrypt password verification and JWT sessions. Student access is bound to the session student ID; mentors can access assigned students and administrators can manage wider records. The backend rechecks role permissions instead of trusting page navigation alone.

11. Evaluation Rubrics - Completion Status

#	Criteria	Distribution	Completion Status
1	Database Design	ER model, relational schema, keys and constraints	Implemented; diagram and 12 relations documented.
2	Database Creation	SQL DDL, NoSQL collection and justification	Implemented; MySQL and MongoDB connections previously verified.
3	SQL Operations and Queries	DML, joins, nested/correlated/recursive SQL	Joins and DML implemented; nested/CTE demo provided. Correlated/recursive query evidence not yet attached.
4	Advanced Database Concepts	Procedure/function, trigger, CTE, NoSQL RUD	Views/indexes implemented; optional trigger/routine creation needs SHOW verification; MongoDB read/upsert present, delete not shown.
5	Web Application and Database Integration	Next.js UI, APIs and DB integration	Implemented and demo screenshots supplied.
6	Authentication and Authorization	Frontend/backend auth and min. two roles	Implemented with three roles: student, mentor, admin.
7	Documentation, Git/GitHub, Demonstration, Testing, and Originality	Reports, repository, individual contribution, originality	Report and tests prepared; GitHub submission and independently authored non-LLM contribution not verified.

11.1 Completed Test Evidence

The observed SQL before/after test is reproducible, and ROLLBACK restores the original readiness. The source contains 12 MySQL table definitions or expected core relations, three views, two composite indexes, three optional audit triggers, one optional procedure, and one optional SQL function. The authorized MongoDB API supports read and update/upsert of feedback.

11.2 Remaining Requirements / Disclosures

- Attach direct MySQL SHOW TABLES and SELECT result screenshots if requested by the evaluator.
- Verify optional stored routines, triggers and advanced query execution on the demonstration machine.
- Supply the actual GitHub repository URL and contribution log, if the faculty rubric requires it.
- Document and demonstrate a genuinely student-designed improvement: the existing rule-based score was AI-assisted and should not be represented as independent non-LLM novelty.
- Verify the course code on the cover: the supplied template originally prints UE23CS351A; this report uses the user-provided UE24CS351A (confirm with faculty).

Conclusion

SkillSync demonstrates a functioning hybrid DBMS architecture with normalized relational data, explainable SQL-derived career recommendations, flexible mentor feedback, authenticated roles, transaction control and administrative reporting. The recorded before/after test supports dynamic calculation and rollback. Full rubric completion remains dependent on the additional verification and independently authored originality evidence noted above.